шаурматч_

Сопроводительная документация

Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника Федерации спортивного программирования (ФСП)

Мероприятие	Хакатон «Лидеры цифровой трансформации 2026», спецтрек ФСП
Команда	«Дергачёв и жена»
Развёрнутый стенд	http://62.113.107.39:8200
Исходный код	https://github.com/IvanDer77/Shaurma
Редакция документа	10.10.2026

Содержание

1. Обзор решения и быстрый старт
2. Функциональная и компонентная архитектура
3. Механика тестирования и подбора: метод «ШАУРМА»
4. Валидация метода: процедура и результаты
5. Обучение моделей на данных платформы («Маринад»)
6. Отчёт симуляции обучения
7. Интеграция с реестром ФСП
8. Описание API
9. Библиотеки и компоненты
10. Сборка, развёртывание и локальный запуск

Соответствие требованиям к документации (ТЗ, п. 3.4)

Требование ТЗ	Раздел документа
Функциональная и компонентная архитектура решения	2
Механика тестирования кандидатов с обоснованием устойчивости к распространению заданий	3 (пункты 5–7 и 11), 4
Механика подбора и логика категоризации кандидатов	3 (пункты 7–9)
Собственная процедура валидации решения и её результаты	4, 5, 6
Схема предполагаемой интеграции с реестром ФСП	7
Описание API	8
Используемые библиотеки и компоненты с указанием версий	9
Пошаговая инструкция по сборке, развёртыванию и локальному запуску	10, 1

Документ собирается из файлов каталога docs/ и спецификации docs/openapi.json скриптом scripts/build_docs_pdf.py, поэтому его содержание совпадает с документацией в репозитории.

1. Обзор решения и быстрый старт

Хакатон «Лидеры цифровой трансформации 2026», спецтрек Федерации спортивного программирования (ФСП).

Кандидат не откликается вслепую. Его категория (**специализация × грейд**) определяется **опросом, выбором грейда и тестом**, который невозможно «слить», — метод **«ШАУРМА»**. Работодатель описывает потребность, получает подборку из подходящих категорий с объяснением и **сам выходит на контакт** с приглашением, где **зарплата «от–до» указана обязательно**. Контакты кандидата открываются только после того, как он принял приглашение.

Посмотреть онлайн

Платформа развёрнута на нашем сервере, ничего устанавливать не нужно:
http://62.113.107.39:8200

Что	Адрес
Платформа (вход)	http://62.113.107.39:8200
Как устроен тест — метод «ШАУРМА» и валидация	http://62.113.107.39:8200/method
Как учится платформа — «Маринад»	http://62.113.107.39:8200/marinade
API: Swagger UI	http://62.113.107.39:8200/api/docs
API: OpenAPI-спецификация	http://62.113.107.39:8200/api/openapi.json
Брендбук	http://62.113.107.39:8200/brand/brandbook.html
Почтовый ящик (письма подтверждения)	http://62.113.107.39:8225

На странице входа справа есть «Быстрый вход»: один клик — и вы в кабинете. Пароль у всех демо-аккаунтов `shaurma2026`.

Что посмотреть, шаг за шагом

1. **Путь кандидата с нуля.** В «Быстром входе» выберите «Новый кандидат». Анкета: согласия, опрос (специализация и стек), резюме и ID ФСП по желанию, выбор грейда и адаптивный тест. После теста — категория вида `BE·MID` и паспорт навыков. Если грейд не подтвердился, сразу предлагается тест на уровень ниже.
2. **Кандидат с историей ФСП.** Войдите как «Алексей Смирнов». Главная: категория, место в категории и сила профиля. «Приглашения»: вилка зарплаты, блок «Почему вы», принять или отклонить с причиной. «Паспорт»: навыки, достижения ФСП, переключатель «Как видит компания», скачивание PDF-профиля. «Задачи»: регулярная задача от компании.
3. **Кандидат без ФСП.** Войдите как «Мария Ковалёва» — профиль без истории ФСП в выдаче не штрафуется.
4. **Работодатель.** Войдите как «ТехноЛаб». «Подбор»: текст потребности разбирается в рецепт навыков, справа — категории и ранжированная выдача с процентом совпадения и

шансом, что кандидат примет приглашение. В карточке кандидата нажмите «Пригласить с зарплатой» (вилка «от–до» обязательна). Далее «Приглашения» (воронка и статусы), «Банк», «Слепая дегустация», «Вакансии» с откликами, «Рынок» (карта категорий).

5. **Регистрация по email.** Нажмите «Нет аккаунта? Зарегистрироваться», письмо с подтверждением придёт в почтовый ящик http://62.113.107.39:8225.
6. **Методы API.** Swagger UI http://62.113.107.39:8200/api/docs: все методы с описаниями и схемами. Чтобы вызывать закрытые методы, в разделе «Аутентификация» выполните `POST /auth/login` (кнопка «Try it out») с телом `{"email": "employer@shaurmatch.demo", "password": "shaurma2026"}` — авторизация хранится в cookie, после этого методы работодателя доступны прямо из Swagger. Для методов кандидата войдите как `alexey@shaurmatch.demo`.
7. **Метод и цифры.** Страница `/method` — шесть частей метода «ШАУРМА» и результаты валидации на 2000 синтетических кандидатах; страница `/marinade` — как модели доучиваются на данных платформы, с интерактивным «годом в перемотке» и полем «попробуйте сами».

Демо-данные можно свободно менять: перед показом мы возвращаем базу в исходное состояние.

Быстрый старт

```
cp .env.example .env          # при необходимости поменяйте порт и PUBLIC_URL
docker compose up -d --build
```

Откройте **http://localhost** (порт задаётся `HTTP_PORT`). При первом старте база заполняется демо-данными: 360 синтетических кандидатов прошли ту же механику ШАУРМА, есть 5 компаний, потребности, вакансии и приглашения.

Демо-аккаунт (пароль `shaurma2026`)	Что показать
`new@shaurmatch.demo`	Путь кандидата с нуля: опрос, тест и категория
`alexey@shaurmatch.demo`	Кандидат BE·SEN с историей ФСП, входящие приглашения
`maria@shaurmatch.demo`	Кандидат FE·MID **без истории ФСП**
`employer@shaurmatch.demo`	Работодатель «ТехноЛаб»: подбор, карта рынка, воронка, «Слепая дегустация»

Можно и зарегистрироваться самому: в демо-режиме (`DEMO_MODE=true`) ссылка подтверждения email показывается на экране. Все письма также видны в Mailpit: `http://localhost:8025`.

Сквозной сценарий (ТЗ, п. 3.3)

1. **Кандидат** регистрируется по email и подтверждает почту, проходит опрос (отрасль, специализация, стек), выбирает грейд и сдаёт адаптивный тест. По его итогам присваивается категория, например `BE·MID`. Если грейд не подтвердился, можно сразу

пройти тест на уровень ниже. Смена грейда — не чаще раза в 90 дней. Можно добавить дополнительные категории (по тесту в другой специализации), например `BE·SEN` + `DS·MID` — у демо-кандидата `alexey@shaurmatch.demo`.

2. **Работодатель** в разделе «Подбор» описывает потребность словами. Система раскладывает текст в рецепт навыков и показывает рекомендованные категории и ранжированную подборку с объяснением. Кандидата приглашают кнопкой «Пригласить», вилка зарплаты обязательна.
3. **Кандидат** в разделе «Приглашения» принимает приглашение или отклоняет его с причиной. После принятия контакты открыты работодателю.
4. **Регулярные задачи:** раз в неделю система адресно предлагает кандидату задачу — сначала от работодателя его категории («Слепая дегустация»), а если таких нет, «Свежую начинку» по «остывшему» или слабому навыку. Решить или предложить подход; проверка автоматическая. **Профиль:** ФИО и места работы видны работодателю по умолчанию (контакты — только после согласия), стек подтверждается заданиями теста, разбор PDF-резюме извлекает роли и места работы, а своя модель подсказывает специализацию.
5. **«Маринад»:** ответы кандидатов калибруют тест, правки рецептов учат языковую модель, ответы на приглашения и отметки «наняли» учат подбор (P(подойдёт) × P(примет)). Всё видно на `/marinade`.
6. **Дополнительно:** вакансии и отклики, регулярные задачи («Слепая дегустация»), «Честный ценник» против фиктивных вакансий, PDF-профиль, режимы доступности, мобильная версия.

Структура репозитория

```
engine/      ШАУРМА-движок (Python, FastAPI): шаблоны, мутации, адаптивный тест, Glicko-2, рецепты, матчинг, антислив, валидация,
             «Маринад» (app/learn: калибровка, детектор дрейфа, ранжировщик, языковая модель; training/: симуляция года)
backend/     REST API (Node.js, Express, TypeScript, PostgreSQL), OpenAPI, авторизация, Keycloak (OIDC)
frontend/    Веб-приложение (React, TypeScript, Vite), дизайн-система в src/styles.css
fsp-mock/    Мок-сервис реестра ФСП (структура данных предложена командой)
keycloak/    Realm «fsp» для входа через ФСП ID (необязательный профиль)
docs/        Документация, отчёт валидации, openapi.json, брендбук и логотипы (docs/brand)
scripts/     Сборка PDF-документации
```

Локальная разработка без Docker

```
docker run -d --name proof-pg -e POSTGRES_USER=proof -e POSTGRES_PASSWORD=proof -e POSTGRES_DB=proof -p 5433:5432 postgres:16-alpine
cd engine   && pip install -r requirements.txt && uvicorn app.main:app --port 8001
cd fsp-mock && npm i && node server.js
cd backend  && npm i && npm run seed && npm run dev      # http://localhost:8080/api/docs
```

```
cd frontend && npm i && npm run dev                        # http://localhost:5173
cd engine   && python -m validation.simulate --n 2000    # пересчитать отчёт валидации
cd engine   && python -m training.marinate --bake         # «год жизни платформы» +
запечь модели «Маринада»
```

Код открыт и не обфусцирован. Компании и демо-профили вымышлены.

2. Функциональная и компонентная архитектура

1. Функциональная архитектура

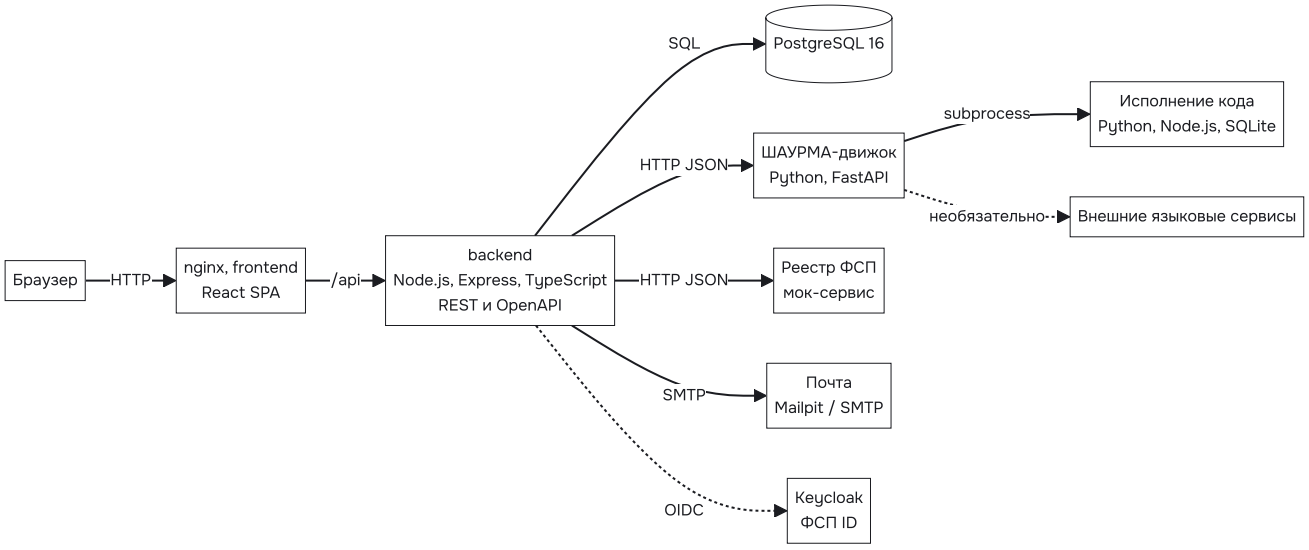
Роль	Функции	Где в коде
Кандидат	Регистрация по email с подтверждением; профиль вручную или из PDF-резюме (авторaспознавание); стандартизированный PDF-профиль; опрос по отрасли и специализации; выбор грейда; адаптивный тест; категория и история; тест на грейд ниже или выше (не чаще раза в 90 дней); входящие приглашения (принять, отклонить с причиной); «Ценник не врёт»; вакансии и отклики; регулярные задачи; приватность и согласия; связь с ID ФСП	`backend/src/routes/candidate.ts`, `tests.ts`, `invitations.ts`, `vacancies.ts`, `tasks.ts`
Работодатель	Профиль компании; описание потребности и рецепт навыков по нему; подборка по категориям с объяснением; уточняющие фильтры поверх подборки; банк кандидатов с фильтрами (специализация, грейд, категория, стек, ФСП); карта рынка; приглашение с вилкой без обязательной вакансии; статусы приглашений; вакансии и отклики; «Слепая дегустация»	`backend/src/routes/employer.ts`, `vacancies.ts`, `tasks.ts`
ШАУРМА-движок	Генерация уникальных вариантов (эталон получается исполнением кода); сложность по LLTM; адаптивный тест и решение по грейду; Glicko-2 по ингредиентам; рецепт по тексту; подбор с объяснением; канарейки и статистика соответствия ответов; разбор резюме; самообучение моделей; валидация	`engine/app/*`, `engine/validation/simulate.py`
Реестр ФСП (мок)	Участник по ID: дисциплины, соревнования, уровень, место, число участников, команда, роль, разряд	`fsp-mock/server.js`

Правила видимости (ТЗ, п. 2.2)

- Зарплата «от–до» в ₽ обязательна в вакансии и в приглашении: это проверки на уровне схемы API (zod) и ограничения `CHECK` в БД.
- Контакты кандидата (email, телефон, Telegram, ФИО) работодатель получает **только** если кандидат принял его приглашение или откликнулся на его вакансию (функция

`contactsVisible` в `services/candidate.ts`). До этого показывается «Кандидат #код». Город и зарплату кандидат может скрыть в настройках.
- Кандидат видит только свои приглашения и отклики: все запросы фильтруются по `candidate_id = req.user.id`.
- Статусы приглашений и откликов: `sent`, затем `viewed`, затем `accepted` или `declined`.
- Выдача формируется только из кандидатов с категорией, присвоенной тестом (`grade IS NOT NULL`) и согласием на публикацию профиля.

2. Компонентная архитектура



- **Монолит с явным разделением слоёв плюс отдельный вычислительный сервис.** Backend отвечает за данные, права и бизнес-правила. Движок не хранит состояния: он только считает, поэтому масштабируется горизонтально.
- **Эталон получается исполнением кода.** Правильный ответ каждого варианта вычисляется запуском сгенерированного кода (Python, Node.js, SQLite) в отдельном процессе с ограничением времени.
- **Хранение.** Реляционная PostgreSQL. Рейтинги Glicko-2 по ингредиентам хранятся в `jsonb` у кандидата, варианты заданий — в `test_items` (это нужно для проверки канареек и аудита).

Модель данных (основное)

`users` связана с `candidates` (1:1) или `companies` (1:1); `candidate_categories` (все подтверждённые тестом категории кандидата: основная + дополнительные; представление `candidate_category_view` — одна строка на кандидата и категорию, по нему ищет работодатель); `test_sessions` и вопросы `test_items`; `grade_history`; `needs` (рецепт `jsonb`); `vacancies` и отклики `applications`; `invitations`; `tasks` и решения `task_submissions` (`tasks.kind`: `employer` — «Слепая дегустация» для категории, `system` — «Свежая начинка», адресная задача кандидату). Схема: `backend/migrations/001_init.sql`.

«Маринад»: обучение на данных платформы

Backend (`services/learning.ts`) решает, когда пора учиться: каждые 300 ответов на задания, 20 ответов на приглашения, после каждой правки рецепта. Он собирает данные из БД и вызывает движок (`/learn/*`). Движок обучает модель и пишет её как открытый JSON в docker-том `models`. Все воркеры подхватывают новую версию по времени изменения файла. Журнал версий хранится в таблице `model_versions`, состояние триггеров — в `learning_state`. Обучение идёт в фоне: пользователь не ждёт. Подробно: МАРИНАД.md.

3. Безопасность и 152-ФЗ

- Пароли хешируются bcrypt. JWT лежит в httpOnly-cookie (SameSite=Lax), также поддерживается `Authorization: Bearer`.
- Роли проверяются на каждом методе (`requireRole`). Все SQL-запросы параметризованы.
- Ограничение частоты запросов на вход, регистрацию и подтверждение email. Используется helmet, CORS разрешён только для `PUBLIC_URL`.
- Согласие на обработку ПДн обязательно перед тестом. Согласие на публикацию обезличенного профиля — отдельно. Дата согласия сохраняется.
- Минимизация данных: работодатель видит категорию, рейтинги и стек без контактов, пока кандидат не дал согласие на контакт.
- Модели. Тест, подбор и разбор текста работают на собственных открытых моделях платформы и не зависят от внешних API. Адаптер внешних языковых сервисов (GigaChat, YandexGPT) необязателен и включается только при наличии ключа: он уточняет веса рецепта и разбор резюме. Контактные данные перед передачей во внешний сервис маскируются.

4. Производительность

- Подборка по потребности строится за один запрос к БД по индексу `(specialization, grade)` и один вызов движка (векторная арифметика по сотням кандидатов). При 360+ кандидатах это десятки миллисекунд.
- Генерация варианта: 1–5 запусков кода, около 50 мс каждый, с LRU-кэшем. Шаг адаптивного теста — EAP на сетке numpy, около 1 мс.
- Движок запускается несколькими процессами uvicorn. Backend и движок масштабируются репликами, так как состояние хранится в PostgreSQL.

5. Интеграции

- **ФСП:** см. ИНТЕГРАЦИЯ ФСП.md.
- **ATS (концепция):** приглашения и статусы публикуются вебхуком (`invitation.created`, `invitation.status_changed`) в ATS компании (Хантфлоу, Potok, E-Staff). Импорт вакансий из ATS идёт по их API. Адаптер — отдельный модуль backend по шаблону `services/fsp.ts`.
- **Защита от фиктивных вакансий («Честный ценник»):** лимит приглашений в неделю (5 + репутация/10), талон возвращается после ответа кандидата. Кандидат подтверждает, совпала ли зарплата с вилкой: «нет» снижает репутацию на 25 и снимает бейдж. Дальнейшее развитие — сигналы фиктивности: вакансия без найма месяцами, одинаковые тексты, вилка-приманка.

6. Развёрнутый стенд и репозиторий

- Стенд для проверки: http://62.113.107.39:8200 (вход по демо-аккаунтам со страницы входа), Swagger — `/api/docs`.
- Исходный код: https://github.com/IvanDer77/Shaurma

3. Механика тестирования и подбора: метод «ШАУРМА»

ШАУРМА — подход платформы «шаурматч_» к тестированию, категоризации и подбору ИТ-специалистов. Документ описывает реализованную механику, её параметры и обоснование устойчивости к распространению заданий. Все числа соответствуют коду в репозитории; результаты проверки приведены в разделе «Валидация метода».

Название — акроним шести частей метода: **Ш**аблоны, **А**даптивная сборка, **У**никальная начинка, **Р**ейтинг, **М**атчинг, **А**нтислив. Образ тоже выбран по существу: задание, как шаурму, собирают заново для каждого кандидата по одному и тому же открытому рецепту.

---

1. Суть метода

В существующих сервисах тест, профиль кандидата и вакансия описаны на трёх разных языках: тест выдаёт балл, резюме — список слов, вакансия — текст. Поэтому подбор сводится к сравнению ключевых слов, а результат теста не участвует в поиске.

ШАУРМА вводит **единый справочник навыков («ингредиентов»)**. Один и тот же ингредиент одновременно:

1. **задаёт сложность задания** — каждый оператор изменения шаблона добавляет известную долю сложности (линейно-логистическая модель теста, LLTM);
2. **измеряется у кандидата** — по каждому ингредиенту ведётся рейтинг Glicko-2 с неопределённостью;
3. **заказывается работодателем** — описание потребности превращается в рецепт: какие ингредиенты нужны и с каким весом.

Задание, кандидат и потребность оказываются в одном пространстве. Тест собирается из тех же ингредиентов, по которым потом ранжируются кандидаты, а объяснение выдачи следует из модели напрямую: «подтверждено 5 из 9 ингредиентов заказа».

2. Аналоги и выводы из них

2.1. Обратный найм

Сервис	Подход	Итог
Hired, Vettery (США)	Компании направляют кандидатам приглашения на интервью с зарплатой	Объединены и в 2024 году свёрнуты в LHH. Проверки навыков не было, модель держалась на перегретом рынке
Triplebyte	Собственный технический скрининг и витрина проверенных инженеров	В 2023 году витрина закрыта, технология тестов продана Karat

Сервис	Подход	Итог
Otta, Cord, Underdog	Профиль с ожиданиями, компании выходят на кандидата	Нет объективной проверки навыков
getmatch, Хабр Карьера, hh.ru	Вакансии с зарплатами, база резюме	Классический отклик, работодатель вручную фильтрует входящий поток

Вывод. Обратный найм без проверенных навыков превращается в рассылку по базе, и площадка теряет кандидатов. Нужна проверка, которая сама поддерживает профиль в актуальном состоянии.

2.2. Проверка навыков

Сервис	Подход	Слабое место
LinkedIn Skill Assessments	Фиксированные тесты с бейджами	Закрыты в 2023 году: ответы ко всем тестам были опубликованы
CodeSignal, HackerRank, Codility	Закрытый банк заданий, ротация, прокторинг	Утечку только замедляют, банк нужно держать в секрете постоянно
Тесты, собранные из библиотеки вопросов по тексту вакансии	Автоматический подбор вопросов	Вопросы фиксированные, результат — общий балл, а не профиль навыков
Автоматическая генерация задания под каждого кандидата	Уникальное задание каждому	Сложность заданий несопоставима, корректность ответа не гарантирована
Codeforces и спортивные рейтинги	Объективный рейтинг	Измеряет только алгоритмическую подготовку и не связан с наймом

2.3. Научные основания

- **Автоматическая генерация заданий (AIG)** — множество вариантов из одной родительской модели.
- **LLTM (линейно-логистическая модель теста)** — сложность задания равна сумме сложностей его операций, поэтому сложность нового варианта известна заранее.
- **Адаптивное тестирование (CAT)** с контролем частоты показа заданий.
- **Glicko-2** — рейтинг с неопределённостью, которая растёт при отсутствии активности.
- **Canary trap и статистика соответствия ответов модели (person-fit)** — выявление списывания.

Новизна метода — в том, что эти блоки работают в одном пространстве ингредиентов, общем для задания, кандидата и потребности, внутри площадки обратного найма.

3. Термины

Термин	Значение
Ингредиент	Элементарный проверяемый навык, например «Граничные случаи» или «SQL JOIN»

Термин	Значение
Лаваш	Открытый шаблон задания с параметрами и точками изменения
Начинка	Набор операторов изменения, применённый к шаблону; определяет уникальный вариант
Рецепт	Потребность работодателя в виде ингредиентов с весами
Вертел	Рейтинг Glicko-2 по каждому ингредиенту, который обновляется после каждого ответа
«Слепая дегустация»	Регулярная задача работодателя, решения которой он видит без имён
«Честный ценник»	Механизм защиты от недобросовестных работодателей
Канарейка	Признак того, что ответ взят из чужого варианта

4. Справочник специализаций, грейдов и ингредиентов

Справочник составлен командой (ТЗ, п. 2). Грейды: **Intern, Junior, Middle, Senior**. Категория кандидата — связка специализации и подтверждённого грейда, например `BE·MID`. Стек и формат работы — фильтры, а не часть категории.

Специализация	Код	Ингредиенты
Backend-разработка	BE	Чтение чужого кода, Граничные случаи, Сложность алгоритмов, Поиск дефектов, Идемпотентность и повторы, Конкурентность и асинхронность, Изменяемое состояние, REST-контракты и пагинация, SQL-агрегации, SQL JOIN
Frontend-разработка	FE	Чтение чужого кода, Граничные случаи, Поиск дефектов, Event loop и промисы, Замыкания и области видимости, Иммутабельность состояния, Типы и приведение в JS, Методы массивов, REST-контракты и пагинация
Data / ML	DS	Чтение чужого кода, Граничные случаи, Сложность алгоритмов, SQL-агрегации, SQL JOIN, Оконные функции, Работа с NULL, Метрики качества моделей, Подготовка данных и утечки

Всего 19 ингредиентов; общие ингредиенты засчитываются во всех специализациях, где они есть. Код: `engine/app/ingredients.py`.

5. Ш — открытый банк шаблонов

- В банке **34 шаблона**: 27 заданий «что выведет код» (Python — 11, JavaScript — 11, SQL — 5) и 7 заданий «найди дефект» (Python — 4, JavaScript — 2, SQL — 1). Шесть шаблонов намеренно лёгкие (сложность от -1,6 до -1,2 логита), чтобы тест точно оценивал уровень ниже Junior.
- У каждого шаблона задана строка ингредиентов, которые он проверяет, и перечень допустимых операторов изменения.
- Все задания имеют четыре варианта ответа и проверяются автоматически.

- Банк можно публиковать открыто: знание шаблона не даёт ответа на конкретный вариант (раздел 6).

Код: `engine/app/templates/`.

6. У — уникальный вариант

Вариант задания = **шаблон + зерно генератора + набор операторов изменения + контекст**.

Слой	Что меняется	Как
Параметры	Размеры и значения входных данных	Детерминированный генератор по зерну
Операторы изменения	Смещение границы на единицу, инвертированное условие, пустой вход, повторный вызов, другая структура данных и другие; всего 72 оператора и 123 допустимых набора	Детерминированный генератор
Контекст	Тот же алгоритм в одной из пяти предметных областей: логистика, банк, соревнования ФСП, поддержка, аналитика	Подстановка имён и формулировки

Правильный ответ получается исполнением сгенерированного кода (Python, Node.js, SQLite), а неверные варианты — исполнением типичных ошибочных версий. Ошибочный эталон исключён по построению.

Сложность варианта известна до показа:

`b(вариант) = b(шаблон) + Σ η(оператор)`

Поэтому разные кандидаты получают разные, но сопоставимые по сложности задания. Начальные значения b и η заданы экспертно и уточняются по накопленным ответам (раздел 12).

Код: `engine/app/generator.py`, `engine/app/runner.py`.

7. А — адаптивный тест и категоризация

Обязательный путь кандидата (ТЗ, п. 2):

1. **Опрос:** отрасль, специализация, стек.
2. **Самостоятельный выбор грейда.**
3. **Адаптивный тест.** - Модель ответа — трёхпараметрическая логистическая (3PL), оценка уровня — ожидаемое апостериорное значение (EAP). - Старт — от центра заявленного грейда. - Следующее задание выбирается по максимуму информации с учётом покрытия ещё не проверенных ингредиентов специализации. - Контроль частоты показа: случайный выбор из пяти лучших заданий и штраф для часто показываемых шаблонов. В проверке ни один шаблон не показывался чаще 4,3% случаев. - Длина теста — **от 8 до 18 заданий**: тест останавливается, когда стандартная ошибка оценки не превышает 0,40.
4. **Решение по грейду:** - вероятность того, что уровень не ниже порога заявленного грейда, не меньше 0,55 — грейд подтверждён (порог подобран на валидации); - вероятность для

следующего грейда не меньше 0,8 — кандидату предлагается тест на уровень выше; - грейд не подтверждён — можно сразу пройти тест на уровень ниже. **Принудительного понижения нет.**

5. **Смена подтверждённого грейда — не чаще одного раза в 90 дней.**
6. **Несколько категорий.** После подтверждения основной категории кандидат может добавить категории в других специализациях, каждую отдельным тестом. Рейтинги общих ингредиентов учитываются во всех категориях. Смена основной специализации не переносит грейд: он появляется только после теста.
7. Побочный результат теста — **профиль ингредиентов** кандидата.

Категория, которую видит работодатель, определяется только этим путём. Данные резюме используются для предзаполнения опроса и на категорию не влияют.

Код: `engine/app/cat.py`, `backend/src/routes/tests.ts`.

8. P — рейтинг по ингредиентам

- Каждый ответ — «матч» кандидата против задания известной сложности. Рейтинг **Glicko-2** ведётся по каждому ингредиенту: сила μ, неопределённость RD и волатильность σ.
- Без активности RD растёт, и консервативная оценка кандидата снижается. Регулярные задачи (раздел 10) возвращают её обратно. Так выполняется требование ТЗ об актуальности профиля.
- **Сила профиля внутри категории** — консервативная оценка `μ - 2·RD`, усреднённая по ингредиентам специализации, плюс надбавка за призовые места ФСП.
- **Достижения ФСП** учитываются как внешние матчи по ингредиентам дисциплины. Сила «соперника» зависит от уровня соревнования: от 1450 (городской) до 1850 (международный). Призовое место или попадание в лучшие 20% засчитывается как победа, в лучшие 50% — как ничья.
- **Отсутствие истории ФСП не снижает позицию.** Достижения только добавляют информацию; профиль без них подтверждается тестами.

Код: `engine/app/glicko.py`.

9. M — подбор и выход на контакт

9.1. Из потребности в рецепт

1. Работодатель описывает потребность обычным текстом.
2. Текст раскладывается на специализацию, грейд, стек, вилку зарплаты и ингредиенты с весами. Разбор выполняют словарь предметной области и собственная текстовая модель платформы (символьные n-граммы, линейные классификаторы). Для каждого ингредиента показывается фрагмент текста, по которому он найден.
3. Работодатель может изменить веса и параметры рецепта; подборка пересчитывается. Правки используются для дообучения модели разбора.

Внешние языковые сервисы для работы не требуются. При наличии ключей GigaChat или YandexGPT их ответ может использоваться для уточнения весов рецепта и разбора резюме; контактные данные в такие сервисы не передаются.

9.2. Ранжирование

Выдача формируется только из кандидатов с категорией, присвоенной тестом, и согласием на публикацию профиля.

```
Совпадение(кандидат, рецепт) = Σ w_i · Освоение_i(μ_i - 2·RD_i)  # ингредиенты рецепта
                             × Соответствие(грейд, стек, формат, зарплата)
                             × (1 + надбавка ФСП, не более 15%)
```

Для ранжирования берётся консервативная оценка `μ - 2·RD` относительно центра требуемого грейда; показываемый процент совпадения считается по `μ - RD` относительно нижней границы грейда. Порядок выдачи определяет двусторонний ранжировщик: **вероятность, что кандидат подойдёт, умноженная на вероятность, что он примет приглашение** с указанной вилкой. Работодатель видит обе величины. Если ранжировщик недоступен, используется формула выше.

9.3. Объяснение выдачи

У каждого кандидата указано, почему он в подборке: сколько ингредиентов рецепта подтверждено, какой из них самый сильный, какие не подтверждены, совпадение стека, давность подтверждения грейда, достижения ФСП, соответствие ожиданий по зарплате.

9.4. Фильтры и банк кандидатов

Фильтры по грейду, стеку и наличию достижений ФСП накладываются поверх сохранённой потребности, подборка при этом не теряется. Отдельно доступен поиск по всему банку кандидатов с фильтрами по специализации, грейду, стеку и достижениям ФСП.

9.5. Приглашение

- Приглашение содержит описание, **вилку зарплаты «от–до» в рублях (обязательно)**, название компании и способ связи. Привязка к опубликованной вакансии не обязательна.
- При составлении приглашения работодатель видит, покрывает ли вилка ожидания кандидата.
- Статусы: отправлено, просмотрено, принято или отклонено. При отказе кандидат указывает причину; причины сводятся в статистику работодателя.
- **Контактные данные кандидата открываются работодателю только после принятия приглашения** или самостоятельного отклика кандидата на вакансию.
- В приглашении кандидат видит, по каким подтверждённым навыкам он выбран.

9.6. Что видит работодатель до согласия кандидата

Категорию, рейтинги ингредиентов, стек (с пометкой, какие технологии подтверждены заданиями теста), достижения ФСП, ожидания по зарплате, опыт из резюме. Имя и места работы показываются, если кандидат не скрыл их в настройках приватности; иначе выводится «Кандидат #код».

9.7. «Честный ценник»: защита от недобросовестных работодателей

- **Недельный лимит приглашений:** `5 + репутация / 10`. Лимит освобождается, когда кандидат ответил на приглашение. Массовая рассылка становится невыгодной.
- **Подтверждение зарплаты:** после принятия приглашения кандидат отмечает, совпала ли зарплата на собеседовании с вилкой. Ответ «нет» снижает репутацию компании на 25 пунктов; при репутации ниже 75 снимается отметка «Честный ценник», которую видят все кандидаты. Ответ «да» повышает репутацию на 2 пункта.

Код: `engine/app/recipe.py`, `engine/app/matching.py`, `engine/app/learn/ranker.py`, `backend/src/routes/employer.ts`, `invitations.ts`.

10. Регулярные задачи

Раз в 7 дней кандидату с подтверждённой категорией адресно предлагается короткая задача.

1. **«Слепая дегустация».** Если есть открытая задача работодателя для категории кандидата, предлагается она. Задача собирается по рецепту потребности; у каждого кандидата свой вариант. Кроме ответа кандидат может описать подход к решению. Работодатель видит решения без имён, с категорией и рейтингом автора, и может пригласить автора с вилкой зарплаты.
2. **«Свежая начинка».** Если задач работодателей нет, система сама подбирает задание по ингредиенту, которому подтверждение нужнее всего: рейтинг давно не обновлялся, ингредиент ещё не проверялся или он слабее остальных. Сложность выбирается немного ниже уровня кандидата.

Результат в обоих случаях обновляет рейтинг ингредиентов. Задача появляется в разделе «Задачи» и приходит письмом.

Код: `backend/src/services/practice.ts`, `engine/app/practice.py`, `backend/src/routes/tasks.ts`.

11. A — защита от распространения заданий

Почему утечка банка не даёт преимущества. 1. Шаблоны открыты: скрывать нечего. 2. Вариант уникален: ответ зависит от зерна и набора операторов. Ответ на чужой вариант к своему не подходит. 3. Сложность вариантов сопоставима (LLTM), поэтому оценки разных кандидатов сравнимы. 4. Эталон получен исполнением кода, поэтому заученный «ответ на шаблон» неверен для изменённого варианта.

Выявление списывания. - **Канарейки.** Если ответ кандидата совпал с правильным ответом чужого варианта того же шаблона, это фиксируется как признак использования чужих ответов. - **Соответствие ответов модели (индекс Iz).** Профиль «трудные задания решены, лёгкие

провалены» статистически маловероятен; при lz ниже -2 попытка помечается. - **Скорость.** Три и более ответов быстрее 6 секунд также помечают попытку.

Помеченная попытка сохраняется в отчёте сессии для проверки; автоматической блокировки нет.

Результаты проверки на модели полной утечки банка приведены в разделе «Валидация метода»: фиксированный тест завышает грейд списывающих на 1,32 уровня, предложенный метод завышения не даёт, канарейки выявляют 91,5% таких попыток при 1,3% ложных срабатываний.

Код: `engine/app/antileak.py`.

12. Самообучение («Маринад»)

Экспертные параметры метода уточняются по данным самой платформы: ответы кандидатов калибруют сложность шаблонов и операторов, правки рецептов дообучают модель разбора текста, ответы на приглашения и итоги собеседований дообучают ранжировщик. Отдельный детектор замечает, что вариант стал аномально лёгким, то есть его разбор распространился. Подробно — в разделе «Маринад».

13. Соответствие требованиям ТЗ

Требование ТЗ	Где реализовано
Категория через опрос, самостоятельный выбор грейда и тест	Раздел 7
Задания устойчивы к распространению	Разделы 5, 6, 11
Сопоставимая сложность при уникальных заданиях	Раздел 6
Формирование заданий на основе описаний вакансий	Разделы 9.1 и 10
Грейд не понижается принудительно, смена ограничена по времени	Раздел 7, 90 дней
Регулярные короткие задания от работодателя	Раздел 10
Ранжирование внутри категории, учёт достижений ФСП	Раздел 8
Выдача по категориям теста с объяснением	Разделы 9.2 и 9.3
Уточнение запроса без потери подборки	Раздел 9.4
Кандидат без истории ФСП	Раздел 8
Выход на контакт, отличный от отклика	Раздел 9.5
Зарплата «от–до» обязательна, контакты скрыты до согласия	Разделы 9.5 и 9.6
Защита от фиктивных вакансий	Раздел 9.7
Собственная процедура валидации	Раздел «Валидация метода»

14. Ограничения и развитие

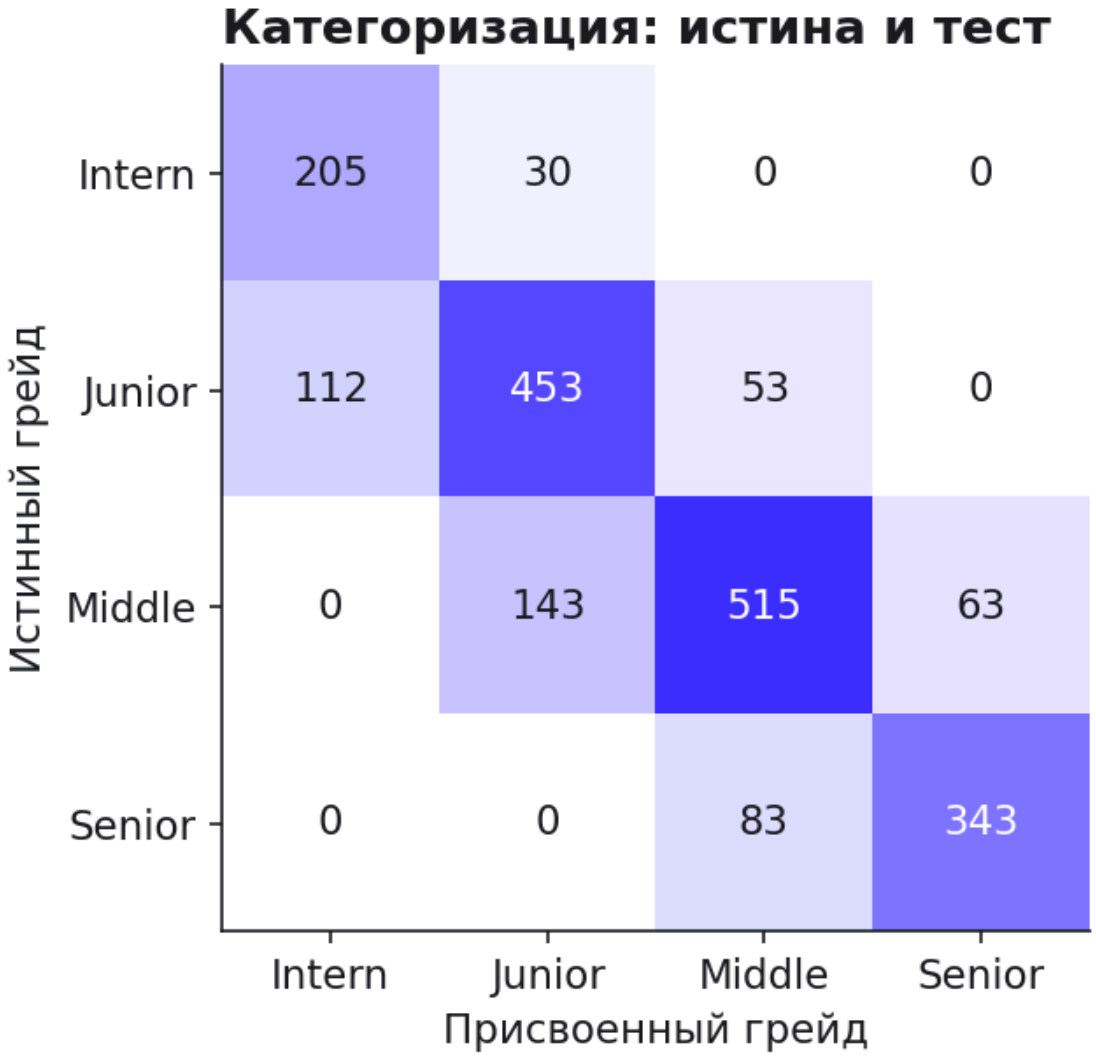
- Банк покрывает три специализации. Метод переносится на другие направления добавлением ингредиентов и шаблонов; для дизайнеров проработан концепт заданий «найди нарушение в макете», где нарушение вносится генератором и потому проверяется автоматически.
- Текст подхода к решению в регулярных задачах пока оценивает работодатель; автоматической оценки нет.
- Проверка помеченных попыток выполняется вручную.
- Интеграция с ATS работодателей и верификация компаний описаны в разделе «Архитектура» как концепция.

4. Валидация метода: процедура и результаты

Синтетическая популяция: **2000 кандидатов** (seed 42) с известными истинными навыками по каждому ингредиенту; самооценка грейда с типичным завышением (30% завышают на уровень). Ответы моделируются по 3PL-модели от истинных навыков ингредиентов задания (многомерная истина при одномерном тесте — проверка устойчивости). Скрипт: `engine/validation/simulate.py`, сырые цифры: `metrics.json`.

1. Категоризация

Метрика	Значение
Точное совпадение грейда с истиной	**76%**
Совпадение ±1 уровень	**100%**
Квадратично-взвешенная к Коэна	**0.87**
Доля завышенных грейдов	7%
Среднее число попыток (включая тест на уровень ниже)	1.31
Восстановление профиля ингредиентов (ρ Спирмена, среднее)	**0.671**



2. Стабильность (тест-ретест на разных уникальных вариантах)

Метрика	Значение
Совпадение грейда при повторном прохождении	**75%**
к (квадратичная) тест-ретест	**0.864**

3. Дискриминативность заданий

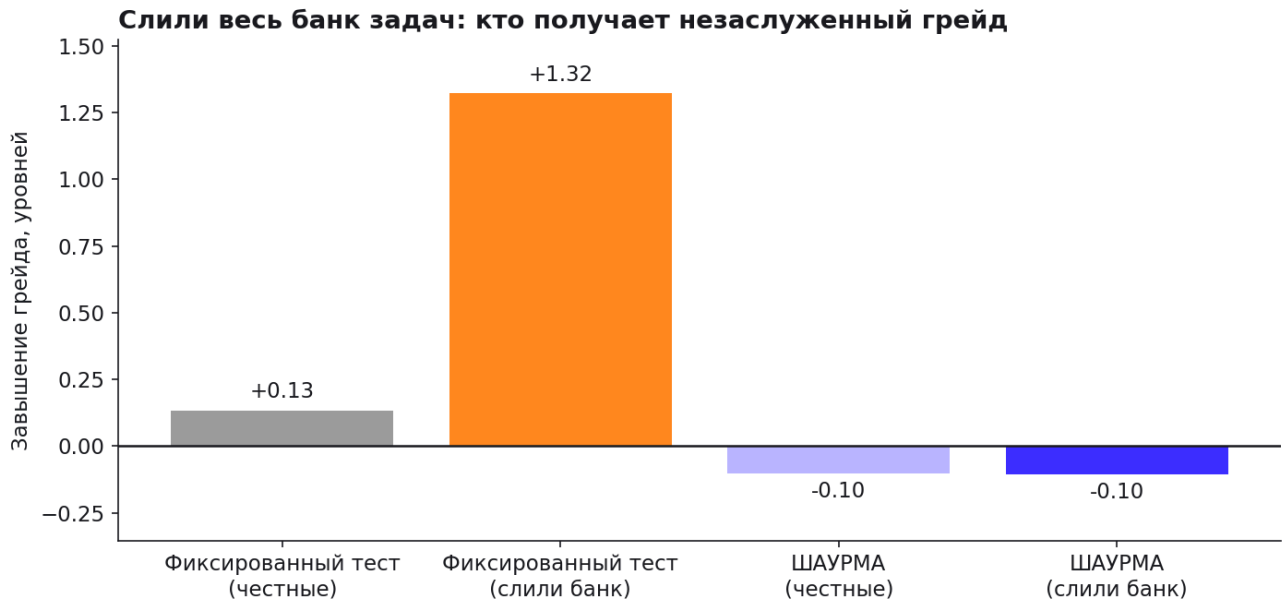
Средняя точечно-бисериальная корреляция между верным ответом и истинным уровнем: **0.341** (ориентир психометрики: > 0.2 — приемлемо, > 0.3 — хорошо). Максимальная доля показов одного шаблона: 4.3%.

4. Устойчивость к сливу

Сценарий: кандидаты знают **весь** банк задач и ответы на чужие варианты.

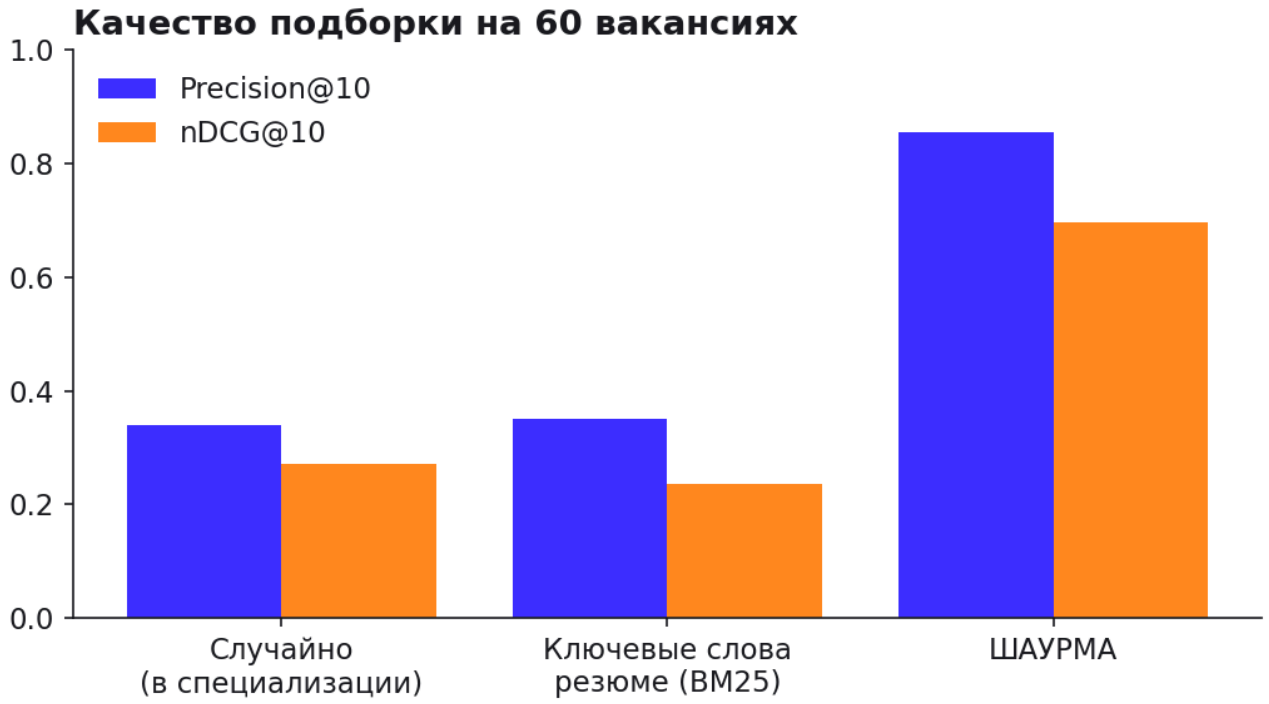
	Завышение грейда
Фиксированный тест, честные	+0.13
Фиксированный тест, слили банк	**+1.32**
ШАУРМА, честные	-0.10
ШАУРМА, слили банк	**-0.10**

Канарейки поймали **92%** списывающих при ложных срабатываниях на честных **1.3%** (порог — 2 совпадения с ответами чужих вариантов; помеченная попытка сохраняется для проверки, автоматической блокировки нет).



5. Подбор (60 вакансий, истинная релевантность известна)

Подход	Precision@10	nDCG@10
Случайно внутри специализации	0.34	0.272
Ключевые слова резюме (BM25)	0.35	0.235
ШАУРМА (категория + ингредиенты + рейтинг)	**0.855**	**0.697**



Ограничения

- Мир синтетический: абсолютные значения зависят от допущений модели ответов. На реальных данных сложности η операторов и дискриминации заданий калибруются заново (LLTM-регрессия по накопленным ответам).
- «Приукрашивание» резюме (30% кандидатов) — допущение; именно от него страдает BM25.

5. Обучение моделей на данных платформы («Маринад»)

В методе «ШАУРМА» есть параметры, заданные экспертно: сложность каждого шаблона и каждой начинки, общая дискриминация заданий, словарь соответствий слов и навыков, веса формулы подбора. «Маринад» заменяет их моделями, которые дообучаются на действиях, уже существующих в продукте. Отдельная разметка, внешние сервисы и сторонние данные для этого не нужны.

Действие в продукте	Что дообучается	Модель	Код
Кандидат отвечает на задание	Сложность шаблонов b0, начинок η, дискриминация a	LLTM + 3PL, MML-EM с априорными распределениями вокруг экспертных значений	`engine/app/learn/calibration.py`
Те же ответы, по времени	Детектор «чёрствого лаваша»: начинка внезапно стала легче	CUSUM по изменению остатков	`engine/app/learn/drift.py`
Работодатель правит рецепт ползунками	Разбор текста потребности в навыки	Своя языковая модель: символьные n-граммы и линейные головы	`engine/app/learn/textmodel.py`
Кандидат принимает или отклоняет приглашение, работодатель отмечает «наняли / не подошёл»	Порядок выдачи подборки	Двусторонний ранжировщик P(подойдёт) × P(примет)	`engine/app/learn/ranker.py`

Посмотреть вживую: страница `/marinade` (интерактивный «год в перемотке», «попробуйте сами», живой паспорт моделей). Полный отчёт с графиками: training/report.md.

Результаты (симуляция «года жизни платформы»)

Мир синтетический, но с известной истиной. Эксперт ошибся в сложностях заданий так же, как ошибаются эксперты в психометрической литературе. У людей есть скрытые навыки и скрытые правила принятия решений. За 12 месяцев приходят 3600 кандидатов и 360 вакансий, накапливается 86 тыс. ответов. В 7-м месяце моделируется распространение разбора начинки «Бинарный поиск: строгое/нестрогое сравнение». Сравнение «до/после» сделано на одних и тех же 2000 новых кандидатах и 150 вакансиях.

	До (эксперт / ручная формула)	После «Маринада»
Ошибка сложности начинок η, RMSE (логиты)	0,307	**0,130**
Ошибка сложности шаблонов b0, RMSE	0,361	**0,138**
Ошибка дискриминации, RMSE log a	0,268	**0,098**
Корреляция оценок η с истиной	0,60	**0,95**
Доля незаслуженно завышенных грейдов	7,4%	**5,2%** (-30%)
Грейд определён точно	77,3%	**79,1%**
Наймов на 10 приглашений	1,90	**2,61** (+37%)
Приглашений принято	49%	**65%**
«Утёкший» приём начинки обнаружен	—	**через 123 ответа** (оценка: легче на 0,58 логита), 0 ложных тревог за год

Языковая модель проверена на 40 описаниях потребностей, написанных вручную отдельно от обучающего корпуса. В них намеренно есть формулировки, которых нет в словаре: «двойные списания», «одно место продаётся двум людям», «трёхзначная логика».

	Словарь правил	Модель шаурматч_
Специализация	90%	**98%**
Грейд	83%	83%
Заказанные навыки в топ-3 рецепта	50%	**68%**

Прирост от правок работодателей пока небольшой. После 30 правок (перекрёстная проверка по 4 фолдам) грейд вырос с 83% до 88%, навыки в топ-3 — с 67% до 68%.

Пересчитать всё: `cd engine && python -m training.marinate --bake` (≈4 минуты, seed 7). Банк — 34 шаблона и 123 набора начинок.

Как это работает в продукте

1. **Калибровка теста.** Каждые `CALIBRATE_EVERY` (по умолчанию 300) новых ответов backend (`backend/src/services/learning.ts`) отправляет все ответы в движок (`POST /learn/calibrate`). - **Страховочный шлюз.** Модель обучается на 80% кандидатов. Новая версия включается, только если на оставшихся 20% она объясняет ответы лучше текущей и лучше экспертной (маргинальное правдоподобие). - **Априорные распределения** держат параметры у экспертных значений, пока данных мало. Чем больше ответов, тем сильнее параметр сдвигается к тому, что показали люди. - **Шкала** привязана к экспертной (mean-mean linking), поэтому пороги грейдов не уплывают. - **В образ калибровка не запекается.**

Она учится только на живых ответах, иначе тест выучил бы сложности вымышленного мира.

2. **Детектор «чёрствого лаваша»** работает перед каждой калибровкой. - Остатки ответов считаются относительно текущей модели. Тревога поднимается, когда остатки начинки растут по сравнению с её собственной историей (k = 0,3, h = 15, разогрев 150 ответов). - Пороги подобраны так, чтобы за год симуляции на ~120 начинках не было ни одной ложной тревоги (максимум CUSUM на честных начинках — 12,9 при пороге 15). - Величина изменения оценивается в момент тревоги: свежее окно ответов против истории до него. - Тревога обнуляет историю начинки: следующая калибровка выучивает её новую сложность.
3. **Ранжировщик** стартует «тёплым»: веса выучены на симуляции. Каждые `RANKER_EVERY` (20) новых ответов на приглашения голова «Примет» дообучается с L2-притяжением к стартовым весам; голова «Подойдёт» — только когда накопится не меньше 50 итогов собеседований (на десятке примеров веса становятся шумом). Если обучен ранжировщик, выдача сортируется по P(найм), а в объяснении появляется строка «вероятно примет приглашение: 72% (мешает: вилка ниже ожиданий)». У работодателя в подборке — оценка «примет ≈N%», в приглашениях — кнопки «Наняли / Не подошёл».
4. **Языковая модель** запечена в образ (8000 сгенерированных текстов + 40 ручных) и переобучается после каждой правки рецепта. Правка весит в обучении в 4 раза больше сгенерированного текста.

Все версии пишутся в журнал `model_versions` (что обучили, на скольких примерах, включили ли и почему). Модели — открытые JSON-файлы: `engine/app/models/` (запечённые) и том `models` (дообученные). Откат к запечённой или экспертной версии: `POST /learn/reset {"name": "calibration" | "ranker" | "textmodel"}`.

API

Метод	Назначение
`GET /api/v1/public/learning`	Паспорт моделей: версии, накопленные сигналы, детектор, журнал
`POST /api/v1/public/learning/recipe`	«Попробуйте сами»: разбор текста в рецепт словарём и моделью (лимит 30/мин)
`POST /api/v1/employer/invitations/{id}/outcome`	Итог собеседования: `hired` / `not_fit`
движок `POST /learn/calibrate`, `/learn/drift`, `/learn/ranker`, `/learn/textmodel`, `/learn/reset`, `GET /learn/status`	Обучение (внутренняя сеть)

Ограничения

- Цифры «года» получены на синтетическом мире: проверен механизм, а не величина эффекта на реальном рынке.

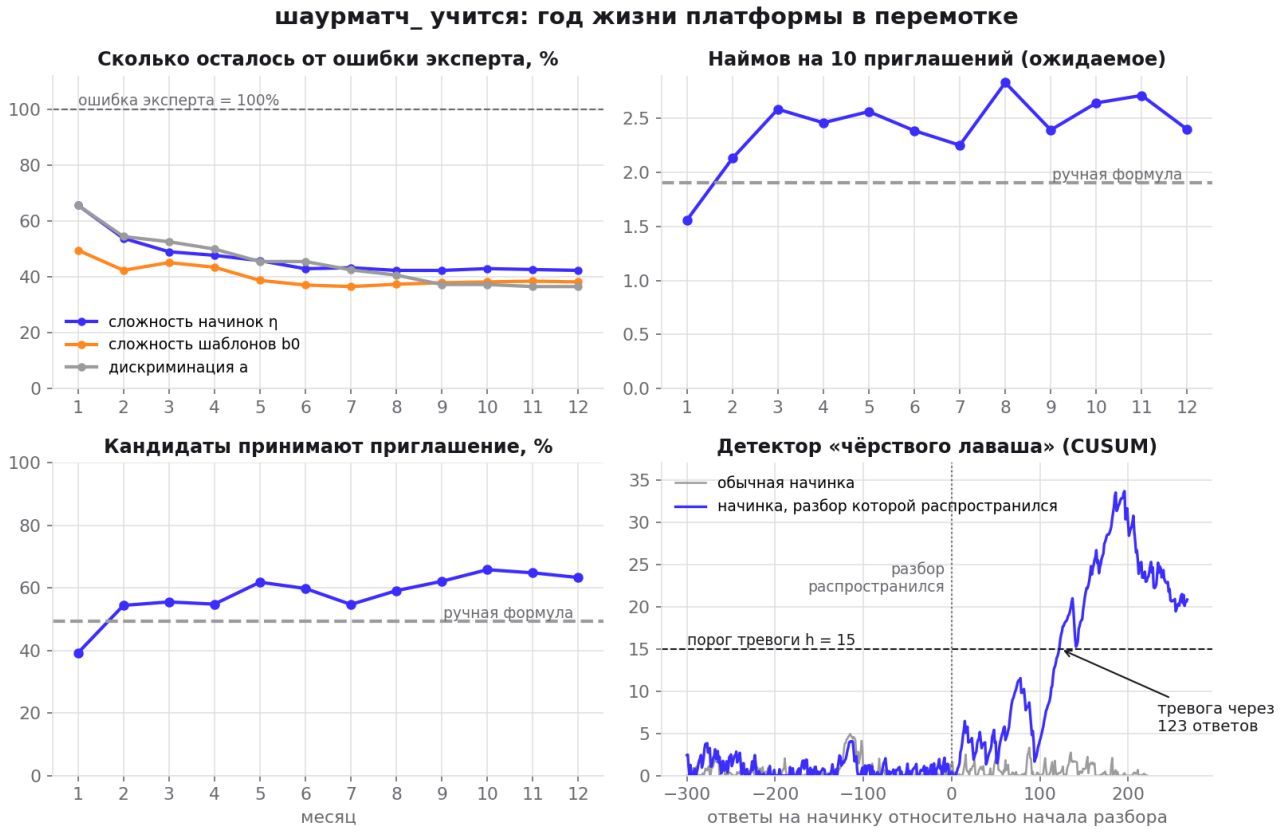
- Ручная выборка для языковой модели маленькая (40 текстов), погрешность около ±7 п. п.
- Ранжировщик учится на приглашениях, которые сам же предложил. Против замыкания на собственном топе в симуляции одно приглашение из пяти — «тёмная лошадка» из топ-30. В продукте выбор остаётся за работодателем.
- Ранжировщик выучил положительный вес у «неопределённости рейтинга». Это поправка к консервативной оценке μ − 2·RD, которая занижает кандидатов с малым числом заданий, а не предпочтение малоизученных кандидатов.

6. Отчёт симуляции обучения

Три модели платформы дообучаются на действиях пользователей в продукте. Отдельная разметка и внешние API для этого не нужны.

Действие в продукте	Чему учит	Модель
Кандидат отвечает на задание теста	Сложность каждой начинки и шаблона, их различающую силу	Калибровка LLTM + 3PL (MML-EM с априорными распределениями вокруг экспертных значений)
Работодатель правит рецепт ползунками	Как текст потребности превращается в навыки	Собственная языковая модель «текст в рецепт» (символьные n-граммы, линейные головы)
Кандидат принимает или отклоняет приглашение, работодатель отмечает найм	Кого приглашать, чтобы найм состоялся	Двусторонний ранжировщик: P(подойдёт) × P(примет)

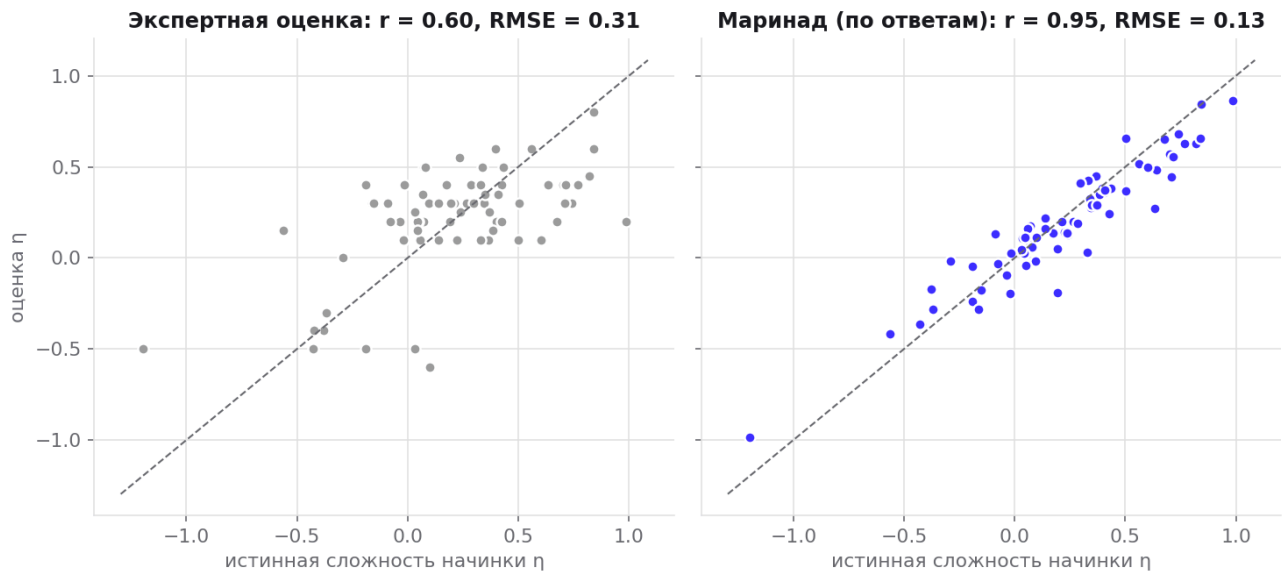
Код: `engine/app/learn/`, симуляция: `engine/training/marinate.py`, сырые цифры: `metrics.json`. Все модели — открытые JSON-файлы с весами, без pickle и обфускации.



1. Год жизни платформы в ускоренной перемотке

Синтетический мир с известной истиной: эксперт ошибся в сложностях заданий примерно так же, как ошибаются эксперты в психометрической литературе, у кандидатов есть скрытые навыки по каждому ингредиенту, а решения «принять приглашение» и «нанять» они принимают по скрытым правилам (деньги, формат, переквалификация, репутация компании, истинное совпадение навыков). Каждый месяц приходят 300 кандидатов и 30 вакансий, а в конце месяца модели дообучаются.

	Месяц 1	Месяц 12
Накоплено ответов на задания	7,481	85,864
Ошибка сложности начинок η, RMSE (эксперт: 0.312)	0.205	**0.13**
Ошибка базовой сложности шаблонов, RMSE (эксперт: 0.361)	0.179	**0.138**
Ошибка дискриминации, RMSE log a (эксперт: 0.268)	0.176	**0.098**
Наймов на 10 приглашений	1.556	**2.393**
Приглашений принято	39%	**63%**

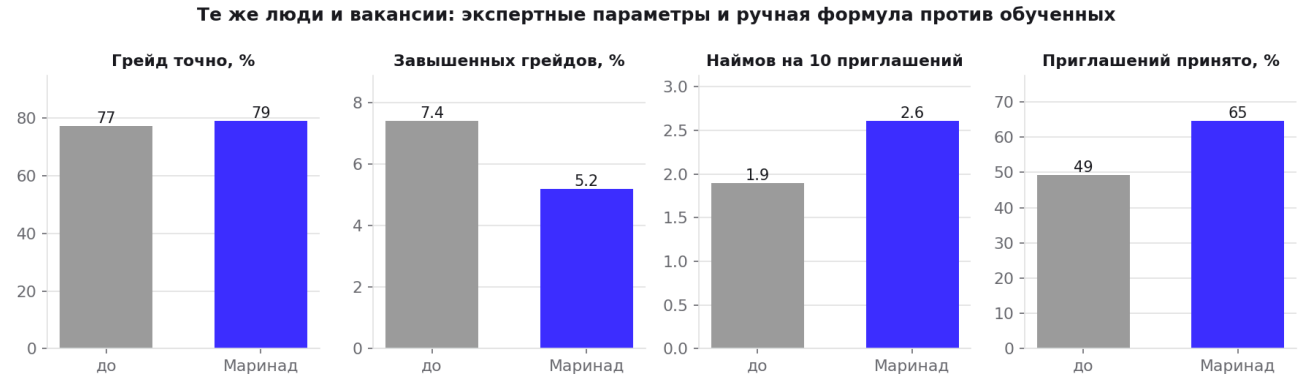


2. До и после на одних и тех же людях

Свежая когорта из 2000 кандидатов проходит тест дважды на одних и тех же случайных числах: с экспертными параметрами и с выученными. 150 новых вакансий ранжируются ручной формулой ШАУРМЫ и обученным ранжировщиком.

Метрика	До (эксперт / формула)	После (Маринад)
Грейд определён точно	77.3%	**79.1%**
к Коэна (квадратичная)	0.875	**0.88**
Доля завышенных грейдов	7.4%	**5.2%**

Метрика	До (эксперт / формула)	После (Маринад)
Наймов на 10 приглашений	1.899	**2.609**
Приглашений принято	49%	**65%**



3. Детектор «чёрствого лаваша»

Банк шаблонов ШАУРМЫ открыт, и ответы не утекают: варианты уникальны. Но может «утечь приём»: разбор приёма начинки распространился, и задание перестаёт различать сильных и слабых. В 7-м месяце так случилось с начинкой «Бинарный поиск: границы: строгое/нестрогое сравнение» (`py_binary_search:upper`): 70% новых кандидатов видели разбор.

Для каждого ответа считается остаток «ответ минус предсказание текущей модели». У любой начинки есть постоянное небольшое несоответствие модели, поэтому детектор ищет не отклонение от нуля, а **изменение**: после разогрева (150 ответов) он копит CUSUM по отклонению остатков от собственной прошлой нормы начинки (k = 0.3, h = 15.0). Детектор поднял тревогу в 7-м месяце, через **123 ответа** на эту начинку после начала разбора, и оценил, что она стала легче на **0.58 логита**. История этой начинки обнулилась, и следующая калибровка выучила её новую сложность. Сложность этой начинки: эксперт +0.35, истинная до разбора -0.19, после разбора в среднем по кандидатам -1.10; «Маринад» к концу года выучил **-0.87**. Ложных тревог на других начинках за год: 0.

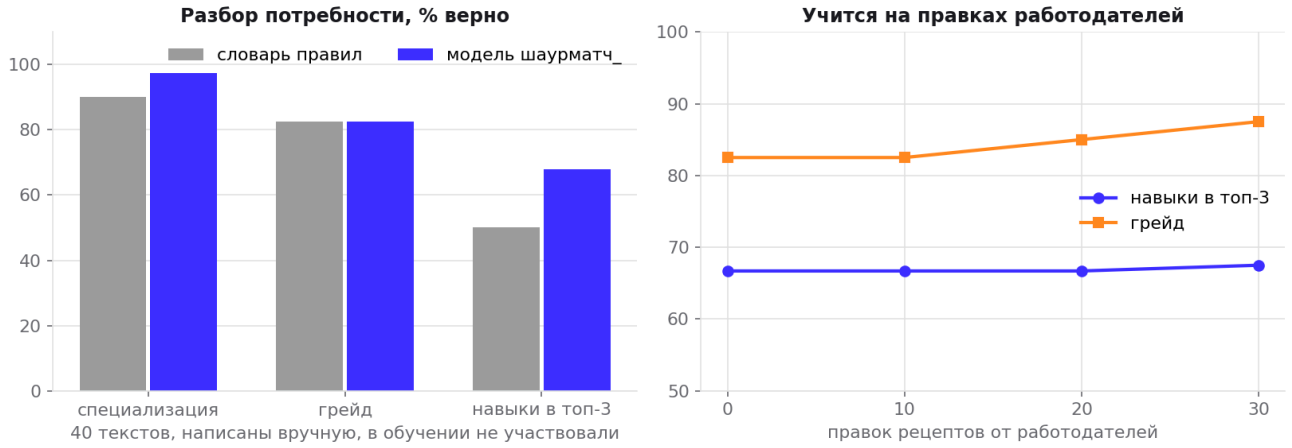
4. Своя языковая модель вместо словаря

Корпус — 8,000 сгенерированных описаний потребностей (банки формулировок по навыкам, грейдам и стекам; парсить чужие вакансии ТЗ запрещает). Проверка — на **40 текстах, написанных вручную отдельно** и генератором не порождаемых: в них много формулировок, которых нет в словаре («двойные списания», «одно место продаётся двум людям», «трёхзначная логика»).

	Словарь правил	Модель шаурматч_
Специализация	90%	**98%**
Грейд	82%	**82%**

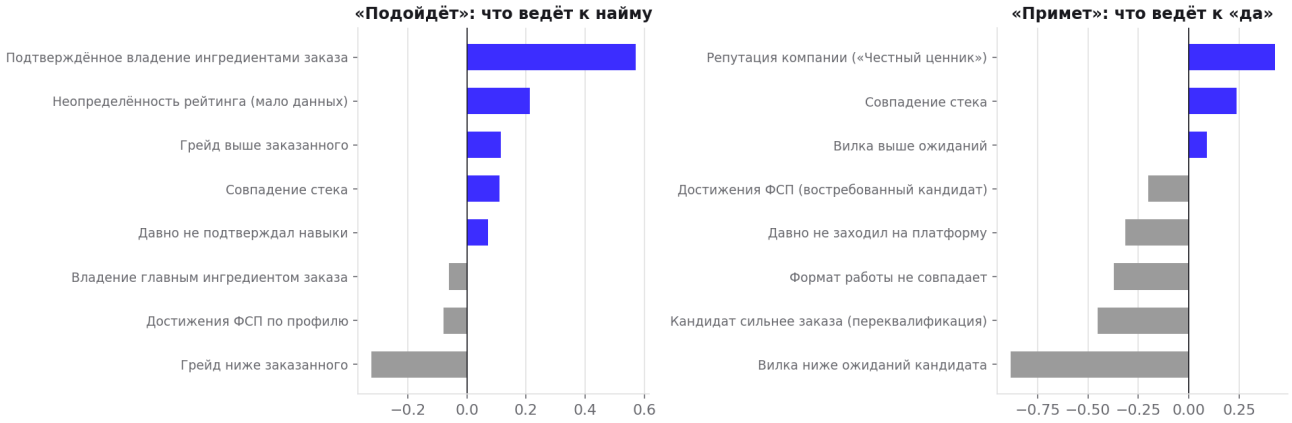
	Словарь правил	Модель шаурматч_
Заказанные навыки в топ-3 рецепта	50%	**68%**

Обучение на правках (4 фолда: модель видит k вручную размеченных текстов как «правки работодателей» и проверяется на остальных): навыки в топ-3 67%, 67%, 67%, 68%, грейд 82%, 82%, 85%, 88% при k = 0, 10, 20, 30.



5. Чему научился подбор

Веса обеих голов открыты и названы по-русски, поэтому подбор остаётся объяснимым: работодатель видит не только «почему кандидат подходит», но и «примет ли он ваше приглашение».



Как это работает в продукте

- **Калибровка теста** учится только на живых ответах: в образ она не запекается, иначе тест выучил бы сложности вымышленного мира. Пока ответов мало, априорное распределение держит параметры у экспертных значений. Пересчёт запускается автоматически каждые 300 новых ответов и пишется в журнал версий (`model_versions`).

- **Ранжировщик** стартует с весов, выученных на симуляции («тёплый старт»), и дообучается на реальных ответах на приглашения с L2-притяжением к стартовым весам.
- **Языковая модель** запечена в образ (корпус + ручные тексты) и переобучается, когда работодатель правит рецепт.
- **Откат** к экспертным параметрам — одна команда; каждая версия модели хранится с метриками.

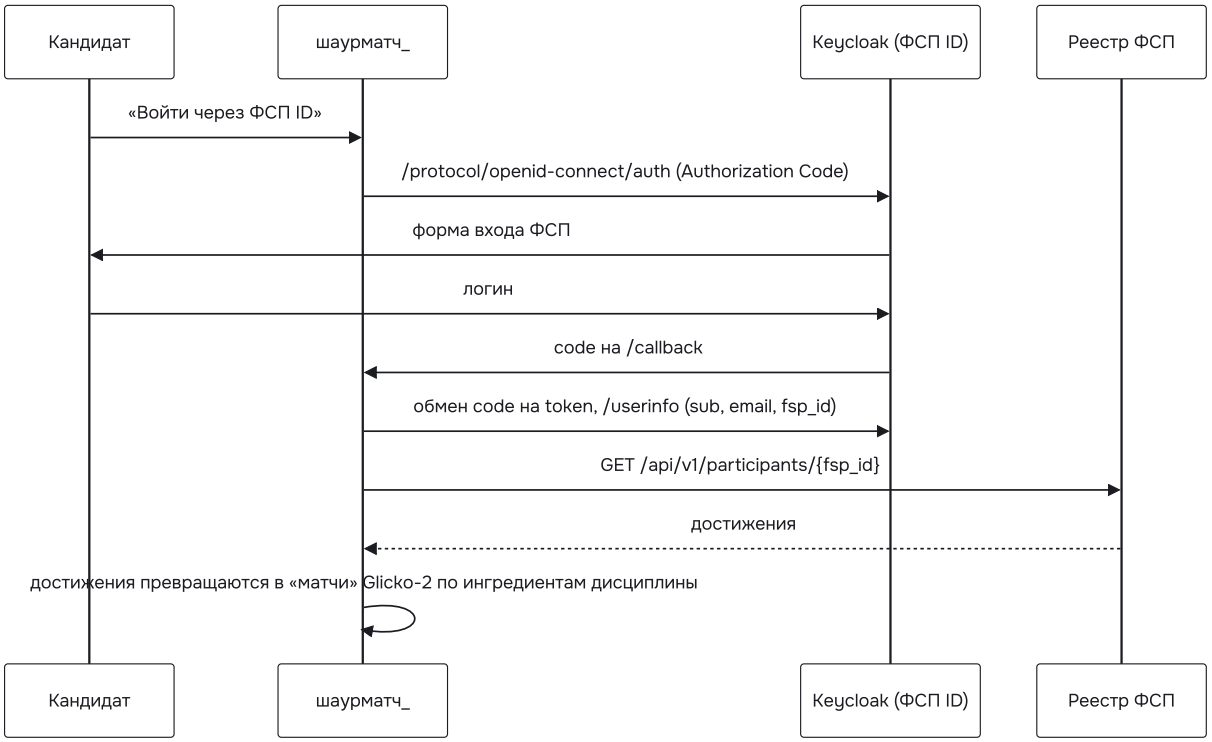
Ограничения

- Мир синтетический: абсолютные цифры зависят от допущений о поведении людей. Механизм проверен, величина эффекта на реальном рынке будет другой.
- Ручная выборка для языковой модели маленькая (40 текстов), поэтому погрешность около ±7 п. п.
- Ранжировщик учится на приглашениях, которые сам же и предложил. Чтобы не замкнуться на собственном топе, одно из пяти приглашений в симуляции — «тёмная лошадка» из топ-30.

7. Интеграция с реестром ФСП

ТЗ (п. 4): API и структура данных ФСП в рамках хакатона не предоставляются, команда сама предлагает, какие сведения получать и как их показывать. Ниже — наше предложение. Оно реализовано в мок-сервисе `fsp-mock/` и в клиенте `backend/src/services/fsp.ts`.

1. Идентификация: ФСП ID на Keycloak



- Реализовано в `backend/src/routes/auth.ts`. Включается переменными `KEYCLOAK_*`; demo-realm лежит в `keycloak/realm-fsp.json`, запуск — `docker-compose.keycloak.yml`.
- Без Keycloak кандидат вводит ID участника вручную: `POST /api/v1/candidate/fsp/link`. Если ID уже привязан к другому профилю, возвращается 409. В промышленной версии привязку нужно подтверждать входом через ФСП ID.

2. Предлагаемая структура данных реестра

`GET /api/v1/participants/{fsp_id}`

```
{
  "fsp_id": "FSP-2025-0001",
  "full_name": "Алексей Смирнов",
  "region": "Москва",
  "achievements": [{
    "event": "Чемпионат России по спортивному программированию 2025",
    "date": "2025-11-20",
    "discipline": "product",
```

```
    "discipline_title": "Продуктовое программирование",
    "level": "national",
    "level_title": "Всероссийский",
    "place": 2,
    "participants": 148,
    "team": "NullPointer",
    "role": "капитан, backend",
    "rank_title": "КМС"
  }]
}
```

Справочники: `GET /api/v1/disciplines`, `GET /api/v1/levels`. Дисциплины соответствуют видам спортивного программирования: алгоритмическое, продуктовое, информационная безопасность, робототехника, беспилотные авиасистемы.

3. Как достижения влияют на подбор

1. Каждое достижение превращается в **«матчи» Glicko-2** по ингредиентам дисциплины (`DISCIPLINE_INGREDIENTS` в `engine/app/glicko.py`). Например, продуктовое программирование даёт REST-контракты, отладку, изменяемое состояние и SQL-агрегации.
2. «Соперник» — рейтинг уровня соревнования: городской 1450 … международный 1850. Результат: призовое место или топ-20% — победа, топ-50% — ничья, остальные — ничья с соперником уровнем ниже.
3. Внутри категории это **повышает силу профиля**: `μ - 2·RD` плюс бонус за призовые места. В подборке достижения ФСП дают небольшой бонус к совпадению (до +15%) и попадают в объяснение («Достижения ФСП: 3, призовых 2»).
4. **Нет истории ФСП — нет штрафа.** Достижения только добавляют информацию. Профиль подтверждается тестами, а в интерфейсе выводится нейтральная пометка «квалификация подтверждена тестами платформы». Фильтр «только с ФСП» работодатель включает сам.
5. **Найм команд (концепция).** Поле `team` позволяет находить кандидатов, выступавших вместе. Это основа будущего группового приглашения.

4. Отказоустойчивость

Если реестр недоступен, запрос возвращает 503 с понятным сообщением, а остальная платформа работает. Достижения кэшируются в профиле кандидата (`fsp_achievements`) и обновляются при повторной привязке. В продакшене — периодической синхронизацией по `fsp_linked_at`.

8. Описание API

Полная спецификация OpenAPI 3: `docs/openapi.json`, интерактивно — Swagger UI на `/api/docs`. Базовый путь: `/api/v1`. Методов: 65. Авторизация — httpOnly-cookie или `Authorization: Bearer`; ошибки — JSON `{error, message, details?}`.

Аутентификация

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/auth/register`	Регистрация по email (с подтверждением адреса)	публично	200, 400, 404
`POST`	`/auth/verify`	Подтверждение email по токену из письма	публично	200, 400, 404
`POST`	`/auth/login`	Вход по email и паролю	публично	200, 400, 404
`POST`	`/auth/resend`	Повторно отправить письмо подтверждения	публично	200, 400, 404
`POST`	`/auth/logout`	Выход	публично	200, 400, 404
`GET`	`/auth/me`	Текущий пользователь (null — не авторизован)	публично	200, 400, 404
`GET`	`/auth/providers`	Доступные способы входа (email, ФСП ID через Keycloak)	публично	200, 400, 404

Кандидат

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/candidate/profile`	Мой профиль	по входу	200, 400, 401, 403, 404
`PUT`	`/candidate/profile`	Заполнить или обновить профиль вручную	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/resume`	Загрузка PDF-резюме с авторазпознаванием (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы) Распознанные данные предзаполняют профиль и опрос. Категорию резюме не определяет — только тест (требование ТЗ).	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/survey`	Опрос по отрасли и специализации + выбор предполагаемого грейда	по входу	200, 400, 401, 403, 404

`PUT`	`/candidate/privacy`	Приватность и согласия (152-ФЗ)	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/fsp/link`	Связать профиль с ID участника ФСП и подтянуть достижения Достижения ФСП засчитываются как внешние подтверждённые «матчи» по ингредиентам дисциплины (рейтинг Glicko-2).	по входу	200, 400, 401, 403, 404
`DELETE`	`/candidate/fsp/link`	Отвязать ID ФСП	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/profile/pdf`	Стандартизированный PDF-профиль (паспорт навыков)	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/grade-history`	История опроса и тестов на грейд	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/market`	Моя категория на рынке: сколько компаний ищут, медиана, моё место	по входу	200, 400, 401, 403, 404

Справочники

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/dict`	Специализации, грейды, ингредиенты, стек, параметры теста	публично	200, 400, 404

Публичное

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/public/stats`	Счётчики для лендинга	публично	200, 400, 404
`GET`	`/public/market`	Открытая карта рынка: кандидаты и медианные ожидания по категориям	публично	200, 400, 404

Работодатель

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/employer/company`	Профиль компании	по входу	200, 400, 401, 403, 404

`PUT`	`/employer/company`	Обновить профиль компании	по входу	200, 400, 401, 403, 404
`POST`	`/employer/needs/parse`	Разобрать описание потребности в рецепт (без сохранения)	по входу	200, 400, 401, 403, 404
`POST`	`/employer/needs`	Описать потребность: какие специалисты нужны, чем занимается команда, стек, грейд	по входу	200, 400, 401, 403, 404
`GET`	`/employer/needs`	Мои потребности	по входу	200, 400, 401, 403, 404
`PUT`	`/employer/needs/{id}`	Поправить рецепт (веса ингредиентов, грейд, стек)	по входу	200, 400, 401, 403, 404
`GET`	`/employer/needs/{id}/selection`	Подборка по потребности: рекомендованные категории + ранжированные кандидаты с объяснением Фильтры применяются поверх сохранённого рецепта — подборка не теряется при уточнении запроса.	по входу	200, 400, 401, 403, 404
`GET`	`/employer/market`	Карта рынка: категории (специализация × грейд), число кандидатов, медиана ожиданий	по входу	200, 400, 401, 403, 404
`GET`	`/employer/candidates`	Банк кандидатов: фильтры по специализации, грейду, категории, стеку, достижениям ФСП; ранжирование внутри категории Внутри категории кандидаты ранжируются по силе подтверждённого профиля: консервативный рейтинг μ − 2·RD по ингредиентам + призовые места ФСП.	по входу	200, 400, 401, 403, 404

`GET`	`/employer/candidates/{id}`	Карточка кандидата (контакты — только после принятия приглашения или отклика)	по входу	200, 400, 401, 403, 404
`GET`	`/employer/stats`	Воронка приглашений и причины отказов	по входу	200, 400, 401, 403, 404

Приглашения

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/employer/invitations`	Пригласить кандидата: описание, зарплата «от–до» (обязательно), способ связи Вакансия не обязательна. Расходует «талон» (антиспам «Честного ценника»).	по входу	200, 400, 401, 403, 404
`GET`	`/employer/invitations`	Мои приглашения и их статусы	по входу	200, 400, 401, 403, 404
`POST`	`/employer/invitations/{id}/outcome`	Итог собеседования по принятому приглашению: наняли или не подошёл Сигнал для «Маринада»: голова «Подойдёт» ранжировщика учится на том, кого в итоге нанимают.	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/invitations`	Входящие приглашения (видны только свои)	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/invitations/{id}/view`	Отметить как просмотренное	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/invitations/{id}/accept`	Принять — контакты раскрываются работодателю	по входу	200, 400, 401, 403, 404

`POST`	`/candidate/invitations/{id}/decline`	Отклонить с причиной в один клик (обратная связь работодателю)	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/invitations/{id}/salary-check`	«Ценник не врёт»: совпала ли зарплата на собеседовании с вилкой приглашения Если нет — у компании падает репутация, урезаются талоны и снимается бейдж «Честный ценник».	по входу	200, 400, 401, 403, 404

Маринад

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/public/learning`	Как платформа учится: активные модели, накопленные сигналы, детектор «чёрствого лаваша», журнал версий Публичный паспорт моделей — прозрачность для кандидатов и работодателей. Кэш 30 секунд.	публично	200, 400, 404
`POST`	`/public/learning/recipe`	Попробуйте сами: разбор текста потребности словарём правил и собственной моделью платформы	публично	200, 400, 404
`GET`	`/public/learning/titles`	Названия шаблонов и начинок	публично	200, 400, 404

Регулярные задачи

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/employer/tasks`	Опубликовать задачу для категории по рецепту потребности	по входу	200, 400, 401, 403, 404
`GET`	`/employer/tasks`	Мои задачи и число решений	по входу	200, 400, 401, 403, 404

`GET`	`/employer/tasks/{id}/submissions`	Обезличенные решения («окошко ларька»): без имени, с результатом и рейтингом	по входу	200, 400, 401, 403, 404
`POST`	`/employer/tasks/{id}/submissions/{sid}/invite`	Пригласить автора понравившегося решения, не зная, кто это	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/tasks`	Мои регулярные задачи: адресные от системы («Свежая начинка») и от работодателей для моей категории Если пора (не чаще раза в 7 дней), система сама создаёт адресную задачу по «остывшему» или слабому ингредиенту.	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tasks/{id}/start`	Получить свой уникальный вариант задачи	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tasks/{id}/submit`	Отправить решение (и подход, если спрашивают)	по входу	200, 400, 401, 403, 404

Тестирование

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/candidate/tests`	Начать адаптивный тест на заявленный грейд Каждому кандидату — уникальные варианты (шаблон + мутации + обёртка) с заранее известной сложностью (LLTM). Правильный ответ получен исполнением кода. specialization — тест в другой специализации: так добавляется дополнительная категория (после подтверждения основной).	по входу	200, 400, 401, 403, 404

`GET`	`/candidate/tests`	Мои тестовые сессии	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/tests/{id}`	Состояние сессии и текущее задание	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tests/{id}/answer`	Ответить на текущее задание	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tests/{id}/abandon`	Прервать тест	по входу	200, 400, 401, 403, 404

Вакансии

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/vacancies`	Опубликованные вакансии	публично	200, 400, 404
`GET`	`/vacancies/{id}`	Вакансия	публично	200, 400, 404
`POST`	`/vacancies/{id}/apply`	Откликнуться на вакансию (контакты раскрываются этой компании)	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/applications`	Мои отклики и их статусы	по входу	200, 400, 401, 403, 404
`GET`	`/employer/vacancies`	Мои вакансии	по входу	200, 400, 401, 403, 404

`POST`	`/employer/vacancies`	Опубликовать вакансию (зарплата «от–до» обязательна)	по входу	200, 400, 401, 403, 404
`PUT`	`/employer/vacancies/{id}`	Редактировать или закрыть вакансию	по входу	200, 400, 401, 403, 404
`GET`	`/employer/vacancies/{id}/applications`	Отклики на вакансию (контакты откликнувшихся открыты)	по входу	200, 400, 401, 403, 404
`POST`	`/employer/applications/{id}/view`	Отметить отклик просмотренным	по входу	200, 400, 401, 403, 404
`POST`	`/employer/applications/{id}/accept`	Принять отклик	по входу	200, 400, 401, 403, 404
`POST`	`/employer/applications/{id}/decline`	Отклонить отклик	по входу	200, 400, 401, 403, 404

9. Библиотеки и компоненты

Всё ПО — с открытыми лицензиями (MIT, Apache-2.0, BSD, PostgreSQL, ISC).

Инфраструктура

Компонент	Версия	Лицензия	Назначение
PostgreSQL	16 (alpine)	PostgreSQL	Реляционная СУБД
nginx	1.27 (alpine)	BSD-2	Раздача SPA, прокси `/api`
Node.js	22	MIT	Backend, мок ФСП, исполнение JS-вариантов
Python	3.12	PSF	ШАУРМА-движок
Mailpit	latest	MIT	SMTP-заглушка и веб-интерфейс писем
Keycloak	26.0	Apache-2.0	ФСП ID (OIDC), по желанию

Backend (`backend/package.json`)

Пакет	Версия	Назначение
express	5.2.1	HTTP-сервер, REST
zod	4.6.5	Валидация запросов
@asteasolutions/zod-to-openapi	9.1.0	Генерация OpenAPI 3 из схем
swagger-ui-express	5.0.1	Swagger UI `/api/docs`
pg	8.23.1	Драйвер PostgreSQL
jsonwebtoken	9.0.3	JWT
bcryptjs	3.0.3	Хеширование паролей
cookie-parser	1.4.7	httpOnly-cookie
helmet	8.3.0	Заголовки безопасности
cors	2.8.6	CORS
express-rate-limit	8.7.0	Ограничение частоты запросов
multer	2.4.0	Загрузка PDF
nodemailer	10.0.15	Письма
pdfkit	0.20.2	PDF-профиль кандидата (шрифты Montserrat и JetBrains Mono, SIL OFL)
typescript	6.0.3	Язык

Пакет	Версия	Назначение
tsx	4.23.15	Запуск TS в разработке

Frontend (`frontend/package.json`)

Пакет	Версия	Назначение
react, react-dom	19.3.0	UI
react-router-dom	7.18.4	Маршрутизация
vite	8.3.2	Сборка
@vitejs/plugin-react	6.1.2	JSX
typescript	7.0.2	Язык
Шрифты Onest, JetBrains Mono	SIL OFL, файлы в `frontend/public/fonts`	Текст, заголовки и числа интерфейса
Шрифт TDAText	фирменный шрифт ФСП из брендбука федерации	Крупные коды категорий и числа

ШАУРМА-движок (`engine/requirements.txt`)

Пакет	Версия	Назначение
fastapi	0.136.3	HTTP API движка
uvicorn	0.49.0	ASGI-сервер
pydantic	2.11.10	Схемы
numpy	2.2.6	EAP-оценка в адаптивном тесте, валидация
pypdf	6.6.0	Извлечение текста из PDF-резюме
python-multipart	0.0.32	Загрузка файлов
httpx	0.28.1	HTTP-клиент адаптера внешних языковых сервисов (необязательный)
matplotlib	3.10.7	Графики отчёта валидации

Методы реализованы самостоятельно, без сторонних психометрических библиотек: Glicko-2 по статье Гликмана, 3PL/EAP, LLTM, индекс lz.

Мок ФСП (`fsp-mock/package.json`)

Пакет	Версия
express	5.2.1

10. Сборка, развёртывание и локальный запуск

Требования

- Docker 24+ и Docker Compose v2. Сервер: 2 vCPU, 4 ГБ RAM, 10 ГБ диска (без Keycloak хватит и 2 ГБ RAM).
- Порт 80 (или любой другой через `HTTP_PORT`) и 8025 для веб-интерфейса почты (по желанию).

Развёртывание на сервере

```
git clone https://github.com/IvanDer77/Shaurma.git && cd Shaurma
cp .env.example .env
#   PUBLIC_URL=http://<ip-или-домен>   — нужен для ссылок в письмах
#   JWT_SECRET=<длинная случайная строка>
#   HTTP_PORT=80
docker compose up -d --build
docker compose logs -f backend      # дождитесь строки «шаурматч_ API: …»
```

Проверка: - `http://<сервер>/` — приложение; - `http://<сервер>/api/docs` — Swagger; - `http://<сервер>/api/health` — `{"status":"ok"}`; - `http://<сервер>:8025` — входящие письма (Mailpit).

HTTPS. Поставьте перед контейнером `frontend` обратный прокси с сертификатом (Caddy, nginx + certbot) и укажите `PUBLIC_URL=https://…`. Cookie станут `Secure` автоматически.

Вход через ФСП ID (Keycloak), по желанию:

```
docker compose -f docker-compose.yml -f docker-compose.keycloak.yml up -d --build
```

Keycloak доступен на `:8180` (admin / `KEYCLOAK_ADMIN_PASSWORD`), демо-пользователь `fsp.demo` / `shaurma2026`. Если сервер не на localhost, задайте `KEYCLOAK_PUBLIC_URL=http://<сервер>:8180`.

Реальная почта. Задайте `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM` и `DEMO_MODE=false`: ссылка подтверждения перестанет показываться на экране.

Внешние языковые сервисы (необязательно). `GIGACHAT_AUTH_KEY` или `YANDEX_API_KEY` и `YANDEX_FOLDER_ID`. Без ключей платформа работает на собственных моделях.

Зеркало пакетов. Если с сервера недоступен `deb.debian.org`, укажите в `.env` зеркало, например `DEBIAN_MIRROR=http://mirror.yandex.ru`: оно используется при сборке образа движка.

Развёрнутый стенд

Рабочий экземпляр для проверки: http://62.113.107.39:8200, почтовый ящик с письмами подтверждения — http://62.113.107.39:8225. Он развёрнут по этой же инструкции с `HTTP_PORT=8200` и `MAIL_UI_PORT=8225`.

Управление данными

```
docker compose exec backend node dist/seed.js --reset   # пересоздать демо-данные
docker compose down -v                                   # удалить всё, включая БД
```

Отключить автонаполнение: `SEED_ON_START=false`.

Локальный запуск без Docker (для разработки)

См. раздел «Локальная разработка» в README. Нужны Python 3.12+ и Node.js 22+ (Node используется движком для исполнения JS-вариантов).

Пересчёт валидации метода

```
cd engine && pip install -r requirements.txt
python -m validation.simulate --n 2000 --seed 42
# результат: docs/validation/report.md, metrics.json, *.png
```

Примечание для Windows

Если путь к проекту содержит кириллицу, `docker compose build` может упасть с ошибкой `x-docker-expose-session-sharedkey`. Запускайте `COMPOSE_BAKE=false docker compose up -d --build`.